BÀI TẬP LÝ THUYẾT
LẬP KẾ HOẠCH VÀ ĐỒ THỊ KẾ HOẠCH TRONG AI

Họ và tên	Trương Hoàng Đăng Khôi
MSSV / Lớp	3124720035 - DNT1241

Mục tiêu học tập

- Mô tả bài toán lập kế hoạch cổ điển bằng trạng thái, mục tiêu, toán tử và các tập Precondition/Add/Delete.
- Xây dựng planning graph theo GraphPlan với các mức trạng thái S_i, mức hành động A_i và các hành động No-op.
- Nhận diện và giải thích các quan hệ mutex: Inconsistent Effects, Interference và Competing Needs.
- Phân tích điều kiện dừng của GraphPlan và trích xuất kế hoạch từ planning graph.
- Vận dụng GraphPlan vào các bài toán Crane, Valet Parking và môi trường planning rời rạc.

Hướng dẫn: Trả lời bằng tiếng Việt, trình bày rõ các bước suy luận. Với câu có planning graph, có thể vẽ trực tiếp lên hình hoặc bổ sung bảng. No-op được tính là một hành động khi xét mutex.

Bài 1. Khái niệm cơ bản trong GraphPlan

Hãy định nghĩa ngắn gọn các khái niệm sau và nêu ý nghĩa của chúng trong việc xây dựng planning graph.

a) Interference (xung đột do can thiệp):

- Định nghĩa: Hai hành động có quan hệ Interference khi một hành động xóa đi tiền điều kiện của hành động còn lại, tức là hiệu ứng của hành động này phủ định tiền điều kiện của hành động kia.
- Ý nghĩa trong planning graph: Giúp loại trừ những cặp hành động không thể thực hiện song song trong cùng một bước thời gian do một hành động phá vỡ điều kiện cần thiết để hành động kia diễn ra.

b) Inconsistent Effects / Inconsistency (hiệu ứng không nhất quán):

- Định nghĩa: Là quan hệ mutex xảy ra khi một hành động tạo ra hiệu ứng phủ định lại hiệu ứng của một hành động khác.
- Ý nghĩa trong planning graph: Đảm bảo tính nhất quán cho tầng trạng thái kế tiếp, loại bỏ các tổ hợp hành động tạo ra kết quả mâu thuẫn trực tiếp với nhau tại cùng một thời điểm.

c) Competing Needs (tiền điều kiện cạnh tranh):

- Định nghĩa: Là quan hệ mutex xảy ra khi tiền điều kiện của hành động A và tiền điều kiện của hành động B có quan hệ mutex với nhau ở tầng trạng thái ngay trước đó (tức là hai điều kiện tiên quyết này không thể đồng thời đúng).
- Ý nghĩa trong planning graph: Giúp lan truyền các ràng buộc xung đột từ tầng trạng thái sang tầng hành động tiếp theo, loại bỏ sớm các hành động không bao giờ có thể kích hoạt cùng lúc do thiếu điều kiện đồng thời.

d) Có những cách nào để tìm một kế hoạch trong mô hình lập kế hoạch cổ điển?

- Tìm kiếm trực tiếp trong không gian trạng thái: Bao gồm tìm kiếm tiến (Forward search / Progression - xuất phát từ trạng thái ban đầu áp dụng các hành động để tiến tới mục tiêu) và tìm kiếm lùi (Backward search / Regression - xuất phát từ trạng thái mục tiêu suy ngược về trạng thái ban đầu).
- Lập kế hoạch thông qua cấu trúc đồ thị / không gian kế hoạch: Sử dụng đồ thị lập kế hoạch để mở rộng xen kẽ các tầng trạng thái và hành động, xác định các quan hệ mutex nhằm thu hẹp không gian tìm kiếm, sau đó trích xuất kế hoạch bằng tìm kiếm ngược khi các mục tiêu đã hội tụ và không xung đột

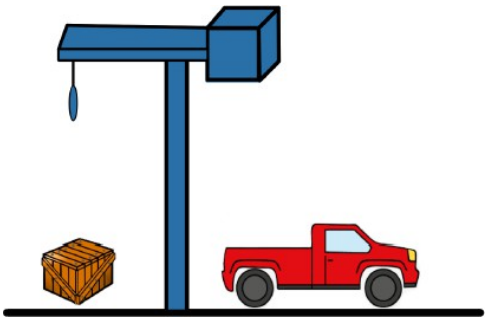
e) Khi dùng heuristic dựa trên GraphPlan, những giả định nào của classical planning được nới lỏng? Vì sao điều này hữu ích hơn so với tìm kiếm ngây thơ?

- Các giả định được nới lỏng:
+ Bỏ qua các hiệu ứng xóa của hành động (giả định thế giới chỉ tích lũy thêm các mệnh đề đúng mà không làm mất đi các mệnh đề cũ).
+ Bỏ qua hoặc nới lỏng các quan hệ loại trừ lẫn nhau (mutex), cho phép nhiều hành động và mệnh đề cùng tồn tại song song ngay cả khi trong thực tế chúng xung đột hoặc chỉ được thực hiện tuần tự.
- Lý do hữu ích hơn tìm kiếm ngây thơ (uninformed/naive search):
+ Đồ thị lập kế hoạch có cấu trúc gọn nhẹ, mở rộng trong thời gian đa thức mà không cần liệt kê toàn bộ không gian trạng thái theo kiểu bùng nổ tổ hợp.
+ Cung cấp hàm ước lượng (heuristic) sát với thực tế về số bước tối thiểu cần thiết để đạt mục tiêu (dựa trên tầng đầu tiên mà các mục tiêu xuất hiện).
+ Giúp cắt tỉa sớm các nhánh tìm kiếm không khả thi nhờ phát hiện các ràng buộc mutex và sớm nhận biết nếu bài toán vô nghiệm.

Bài 2. Lập kế hoạch ký hiệu - Bài toán cần cẩu

Cho một cần cẩu, một kiện hàng và một xe tải. Ban đầu kiện hàng ở bên trái, xe tải ở bên phải và cần cẩu quay về bên trái. Mục tiêu là đưa kiện hàng lên xe tải và cuối cùng cần cẩu quay về bên trái.

Cần cẩu có thể quay giữa trái và phải, có hoặc không mang kiện hàng. Cần cẩu có thể nhấc kiện hàng nếu cả hai ở cùng phía. Kiện hàng chỉ có thể được đưa lên xe tải bằng cần cẩu.



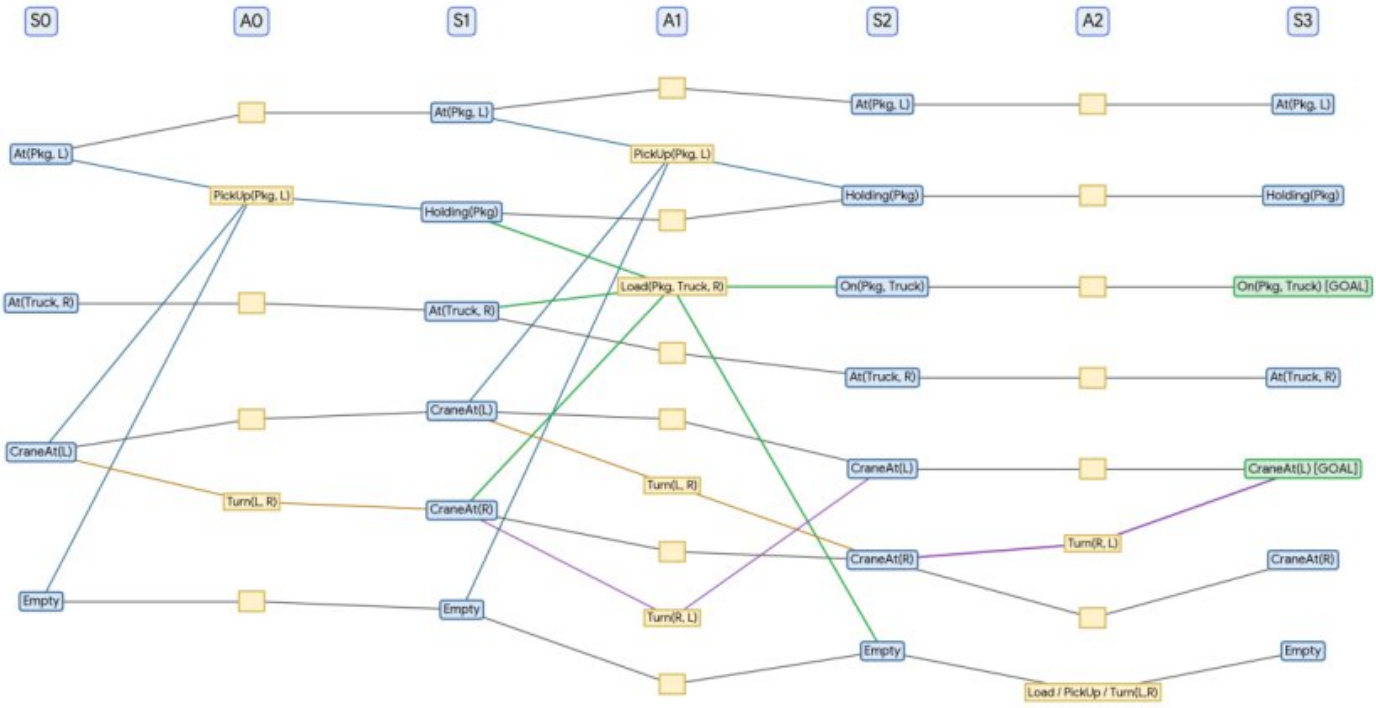
a) Xác định các biến/đối tượng, các predicate cần dùng, trạng thái ban đầu và trạng thái mục tiêu.

- **Đối tượng / Hằng số:** C (cần cẩu), B (kiện hàng), T (xe tải), Left (bên trái), Right (bên phải).
- **Các Predicate cần dùng:**
  - **At(x, p):** Đối tượng x (kiện hàng B hoặc xe tải T) đang nằm ở vị trí p ∈ {Left, Right}.
  - **Facing(C, p):** Cần cẩu C đang quay về phía p ∈ {Left, Right}.
  - **Holding(C, B):** Cần cẩu C đang nhấc/giữ kiện hàng B.
  - **Empty(C):** Móc cần cẩu C đang trống (không giữ hàng).
  - **In(B, T):** Kiện hàng B đã được đặt lên xe tải T.
- **Trạng thái ban đầu (S_0):** At(B, Left) ∧ At(T, Right) ∧ Facing(C, Left) ∧ Empty(C)
- **Trạng thái mục tiêu (Goal):** In(B, T) ∧ Facing(C, Left)

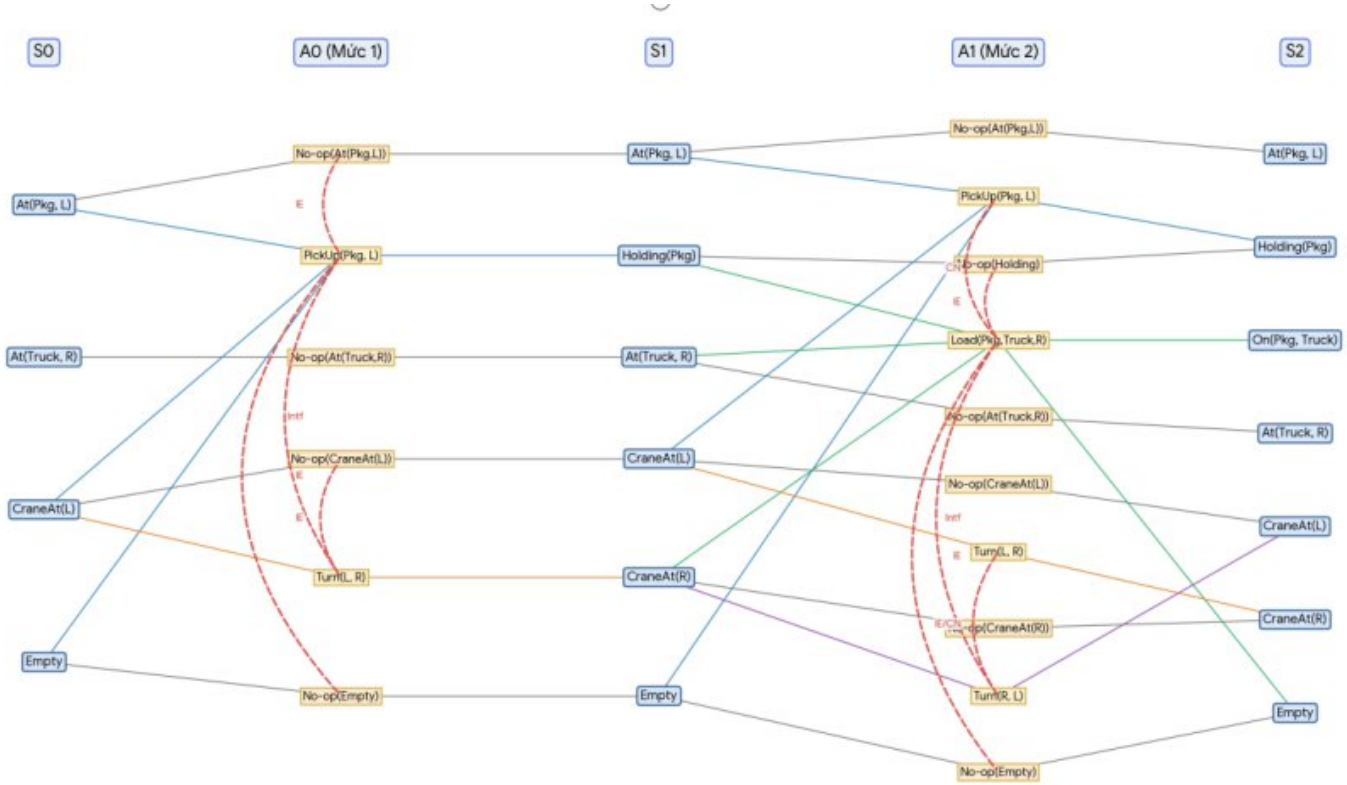
b) Xây dựng các toán tử hành động. Với mỗi toán tử, ghi rõ Preconditions, Add List và Delete List.

Toán tử	Preconditions	Add List	Delete List
Turn(p_1, p_2) (Quay từ p_1 sang p_2, $p_1 \neq p_2$)	Facing(C, p_1)	Facing(C, p_2)	Facing(C, p_1)
PickUp(p) (Nhấc kiện hàng B tại phía p)	Facing(C, p) ∧ At(B, p) ∧ Empty(C)	Holding(C, B)	At(B, p), Empty(C)
PutDown(p) (Đặt kiện hàng B xuống đất tại phía p)	Facing(C, p) ∧ Holding(C, B)	At(B, p), Empty(C)	Holding(C, B)
Load(p) (Đưa kiện hàng B lên xe tải T tại phía p)	Facing(C, p) ∧ At(T, p) ∧ Holding(C, B)	In(B, T), Empty(C)	Holding(C, B)

c) Vẽ planning graph cho 3 bước đầu tiên. Có thể dùng hình minh họa thay cho proposition nếu biểu diễn rõ ràng.



d) Vẽ hai mức đầu tiên của GraphPlan và chỉ ra các cặp hành động mutex; với mỗi cặp, ghi loại mutex.

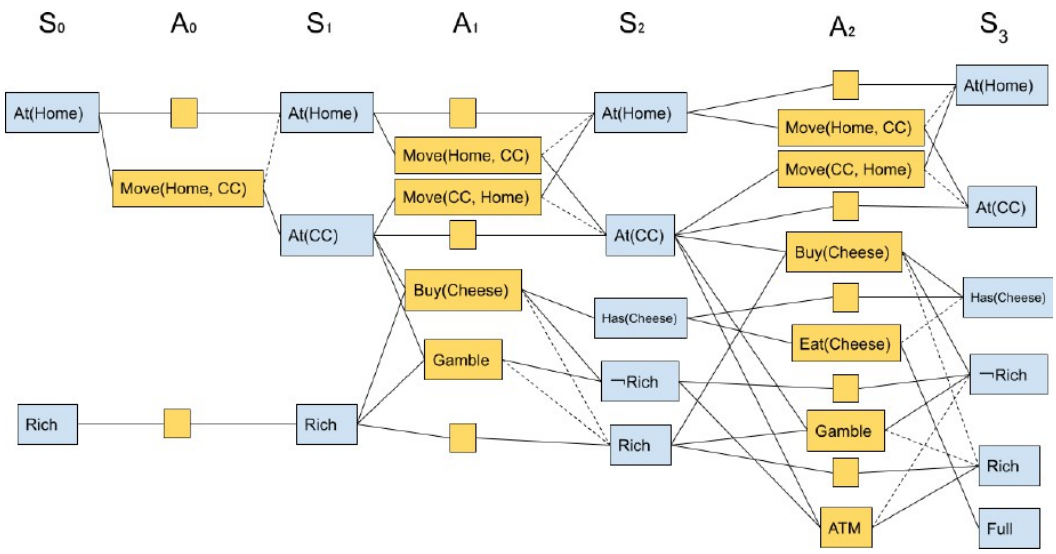


Bài 3. Quan hệ Mutex trong planning graph

Pinky đi lấy thức ăn. Các hành động được cho như sau:

Hành động	Preconditions	Add List	Delete List
Move(A,B)	At(A)	At(B)	At(A)
Buy(Cheese)	At(ChuckyCheese), Rich	Has(Cheese), ¬Rich	
Gamble	At(ChuckyCheese), Rich	¬Rich	Rich
ATM	At(ChuckyCheese), ¬Rich	Rich	¬Rich
Eat(Cheese)	Has(Cheese)	Full	Has(Cheese)

Trạng thái bắt đầu: Rich ∧ At(Home). Mục tiêu: Full. Planning graph tương ứng được cho trong hình dưới:



Planning graph cho bài toán Pinky.

a) Ở mức A_0, hãy chỉ ra một cặp hành động mutex do Inconsistent Effects.

Cặp hành động: Move(Home, CC) và No-op(At(Home))

b) Ở mức A_1, hãy chỉ ra một cặp hành động mutex do Interference.

Cặp hành động: Buy(Cheese) và Gamble

c) Ở mức A_2, hãy chỉ ra một cặp hành động mutex do Competing Needs.

Cặp hành động: Buy(Cheese) và ATM

d) Giải thích ngắn gọn vì sao mỗi cặp ở trên thỏa loại mutex tương ứng.

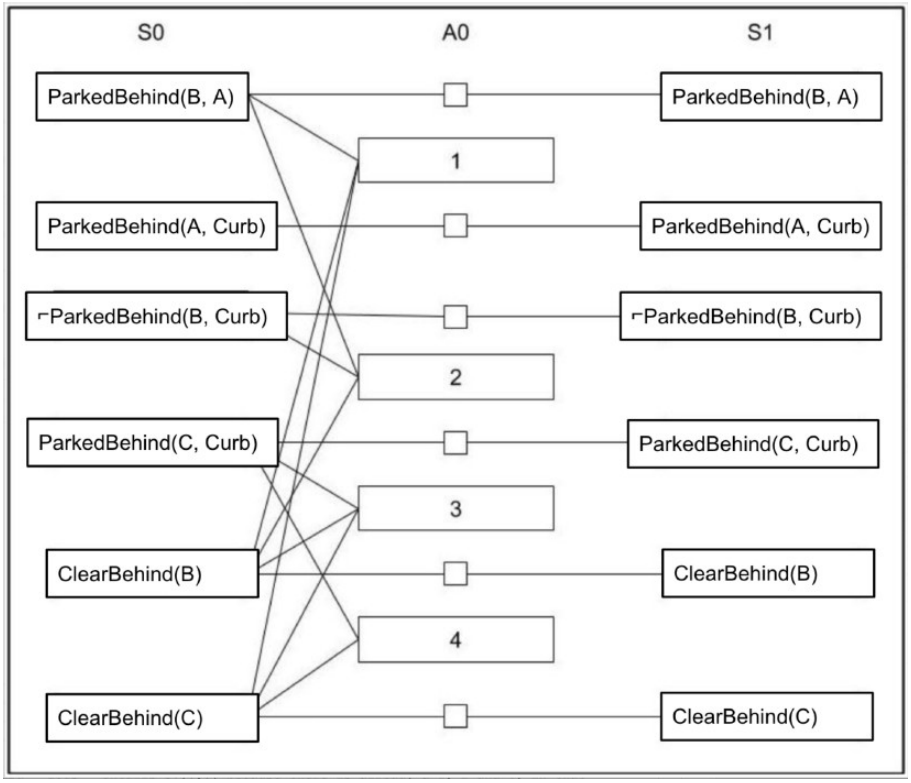
- Mức A0: Move(Home, CC) và No-op(At(Home)): Hành động Move(Home, CC) có hiệu ứng xóa At(Home), trái ngược trực tiếp với hiệu ứng duy trì $(\neg \text{At(Home)}$ của hành động No-op(At(Home)).
- Mức A1: Buy(Cheese) và Gamble (Interference): Hành động Gamble xóa tiền điều kiện Rich của Buy(Cheese) (và ngược lại, Buy(Cheese) thêm hiệu ứng ¬Rich làm phủ định tiền điều kiện Rich của Gamble), khiến hai hành động can thiệp và phá hủy điều kiện thực hiện của nhau.
- Mức A2: Buy(Cheese) và ATM (Competing Needs): Buy(Cheese) cần tiền điều kiện Rich, trong khi ATM cần tiền điều kiện ¬Rich. Ở tầng trạng thái S2 ngay trước đó, hai mệnh đề Rich và ¬Rich có quan hệ mutex với nhau (do là phủ định của nhau), nên hai hành động này không thể cùng được kích hoạt.

Bài 4. Classical Planning và GraphPlan - Valet Parking

Xét bài toán Valet Parking với hai predicate ClearBehind(car) và ParkedBehind(car1, car2). Hai toán tử được định nghĩa như sau:

Thành phần	**ParkBehind(car1, car2)**	**ParkInNewRow(car)**
Preconditions	ClearBehind(car1) ClearBehind(car2) ParkedBehind(car1, place)	ClearBehind(car) ParkedBehind(car, place) ¬ParkedBehind(car, curb)
Add List	ParkedBehind(car1, car2) ClearBehind(place) ¬ParkedBehind(car1, place) ¬ClearBehind(car2)	ParkedBehind(car, curb) ClearBehind(place) ¬ParkedBehind(car, place)
Delete List	ClearBehind(car2) ParkedBehind(car1, place)	ParkedBehind(car, place) ¬ParkedBehind(car, curb)

Hình duowis là mẫu cho hai mức đầu tiên $S_0 \rightarrow A_0 \rightarrow S_1$. Các nút hành động No-op đã được biểu diễn bằng các ô vuông nhỏ không tên.



Mẫu GraphPlan cho Valet Parking

a) Điền tên hành động phù hợp vào các ô 1-4 ở mức A_0. Nếu không tồn tại hành động hợp lệ cho tập tiền điều kiện đó, ghi "N/A".

- Ô 1: ParkBehind(B, C) (nối với ParkedBehind(B, A), ClearBehind(B), ClearBehind(C)).
- Ô 2: ParkInNewRow(B) (nối với ParkedBehind(B, A), ¬ParkedBehind(B, Curb), ClearBehind(B)).
- Ô 3: ParkBehind(C, B) (nối với ParkedBehind(C, Curb), ClearBehind(B), ClearBehind(C)).
- Ô 4: N/A

b) Với từng hành động 1-4, liệt kê TẤT CẢ predicate ở S_1 nhận add-edge từ hành động đó. Nếu hành động không hợp lệ, ghi "N/A".

- Hành động 1: ParkBehind(B, C): ParkedBehind(B, C), ClearBehind(A), ¬ParkedBehind(B, A), ¬ClearBehind(C).
- Hành động 2: ParkInNewRow(B): ParkedBehind(B, Curb), ClearBehind(A), ¬ParkedBehind(B, A).
- Hành động 3: ParkBehind(C, B): ParkedBehind(C, B), ClearBehind(Curb), ¬ParkedBehind(C, Curb),
- Hành động 4: N/A.

c) Nêu hai action node có quan hệ mutex do Inconsistent Effects. No-op được phép sử dụng.

Cặp hành động: ParkBehind(B, C) và No-op(ParkedBehind(B, A)).

d) Nêu hai action node có quan hệ mutex do Interference.

Cặp hành động: ParkBehind(B, C) và ParkBehind(C, B).

e) GraphPlan "leveled off" khi nào? Chọn một đáp án:

☐ A. Tất cả các hành động có thể đã được khám phá.

☐ B. Không còn tập literal khác rỗng nào không có liên kết mutex.

☒ C. Hai mức liên tiếp của planning graph giống hệt nhau.

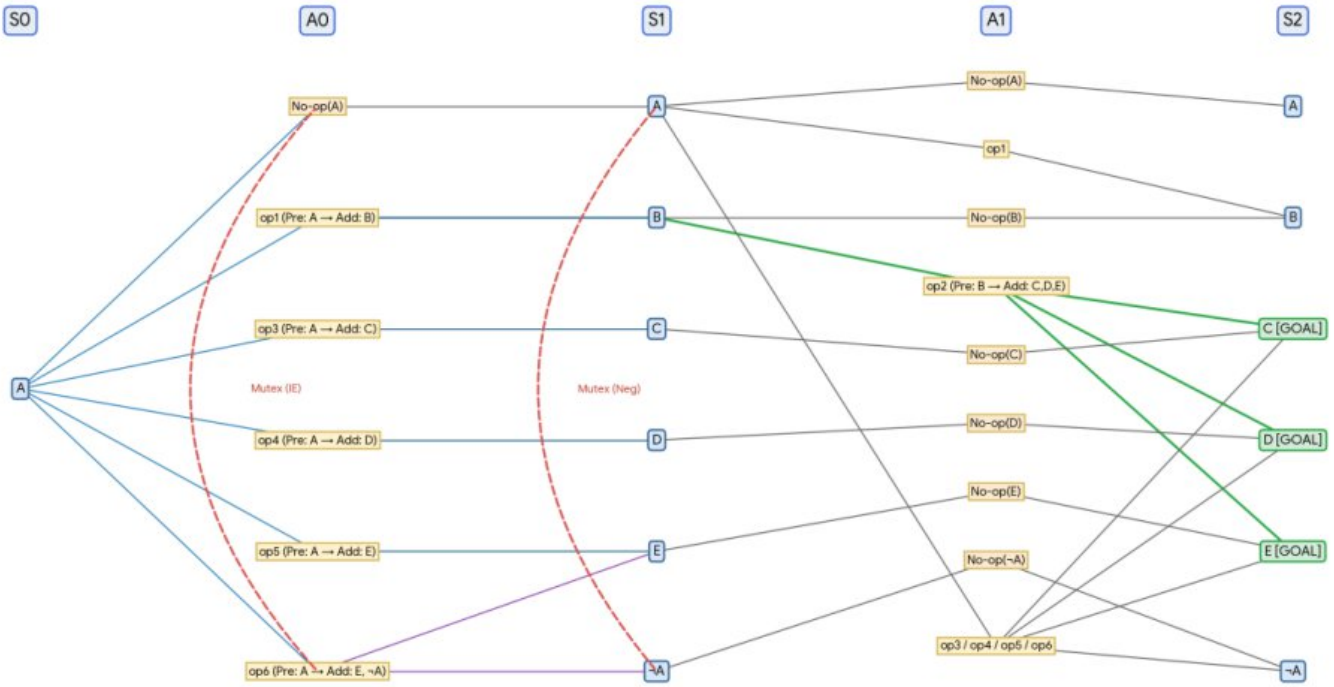
☐ D. Mức trạng thái cuối cùng chứa một trạng thái mục tiêu.

Bài 5. Xây dựng GraphPlan và trích xuất kế hoạch

Cho môi trường lập kế hoạch với 6 toán tử. Trạng thái bắt đầu là A; mục tiêu là C ∧ D ∧ E. Mỗi thời điểm chỉ áp dụng một toán tử và cần tìm kế hoạch có ít thao tác nhất.

	op1	**op2**	**op3**	**op4**	**op5**	**op6**
Precondition	A	B	A	A	A	A
Add	B	C, D, E	C	D	E	E, ¬A
Delete						

a) Chạy GraphPlan: vẽ lần lượt các mức trạng thái và mức hành động cho đến khi thuật toán dừng. Nhớ bổ sung các No-op cho trạng thái cần duy trì.



b) Kế hoạch nào được GraphPlan trả về?

Kế hoạch trả về gồm 2 bước: [op1 → op2]

• Bước 1 (tại A_0): Thực hiện op1 (Pre: A → Add: B) để tạo ra mệnh đề B ở S_1.

• Bước 2 (tại A_1): Thực hiện op2 (Pre: B → Add: C, D, E) để tạo ra đồng thời cả ba mệnh đề mục tiêu C, D, E ở S_2.

c) Kế hoạch trên có tối ưu về số lượng thao tác hay không? ☒ Có ☐ Không

Giải thích: Vì đề bài giới hạn mỗi thời điểm chỉ được áp dụng một toán tử duy nhất:

• Với 1 thao tác duy nhất, không có toán tử nào từ trạng thái ban đầu A sinh ra đồng thời cả 3 mục tiêu C, D, E (op3, op4, op5, op6 mỗi toán tử chỉ tạo ra 1 mục tiêu riêng lẻ).

• Nếu dùng các toán tử trực tiếp op3, op4, op5 thì cần tới 3 thao tác tuần tự. Trong khi đó, chuỗi [op1 → op2] chỉ mất đúng 2 thao tác, nên đây là số lượng thao tác ít nhất (tối ưu).

d) Liệt kê TẤT CẢ cặp toán tử mutex ở A_0 và TẤT CẢ cặp proposition mutex ở S_1. Nếu không có, ghi "Không có".

Tầng hành động A_0 (Cặp toán tử mutex): op6 ↔ No-op(A)

Tầng trạng thái S_1 (Cặp proposition mutex): A ↔ ¬A

Bài 6. Tổng hợp và phân tích

a) So sánh tìm kiếm trực tiếp trong không gian trạng thái với GraphPlan. Trong trường hợp nào planning graph giúp giảm đáng kể không gian tìm kiếm?

- Tìm kiếm trực tiếp trong không gian trạng thái: Duyệt qua từng trạng thái cụ thể, dễ bị bùng nổ tổ hợp khi số lượng hành động có thể thực hiện song song hoặc độc lập tăng cao.
- GraphPlan: Không liệt kê từng trạng thái riêng lẻ mà nén toàn bộ các mệnh đề và hành động khả dĩ vào các tầng xen kẽ (S_i, A_i) trong thời gian đa thức, đồng thời dùng các ràng buộc mutex để loại bỏ sớm các tổ hợp xung đột trước khi trích xuất kế hoạch.
- Trường hợp giảm đáng kể không gian tìm kiếm: Các miền bài toán kiểu STRIPS có không gian trạng thái lớn, nhiều hành động độc lập có thể thực hiện song song hoặc chứa nhiều nhánh xung đột (nhờ mutex cắt tỉa sớm và phát hiện nhanh nhánh thất bại).

b) Vì sao No-op quan trọng trong GraphPlan? Nếu bỏ toàn bộ No-op thì việc biểu diễn trạng thái tồn tại qua nhiều mức sẽ bị ảnh hưởng như thế nào?

- Vai trò của No-op: Đại diện cho việc "không làm gì" nhằm duy trì một mệnh đề đã đúng ở tầng S_i tiếp tục tồn tại sang tầng S_{i+1}, đồng thời giúp phát hiện xung đột khi một hành động khác ở tầng A_i cố tình xóa mệnh đề đang được duy trì đó.
- Hậu quả nếu bỏ toàn bộ No-op: Các mệnh đề đã đúng ở tầng trước nhưng không được hành động nào ở tầng hiện tại tạo lại sẽ bị biến mất khỏi tầng trạng thái kế tiếp ($S_{i+!}$), làm mất tính liên tục của thế giới và khiến các hành động ở bước sau không còn đủ tiền điều kiện để kích hoạt.

c) Cho một mục tiêu gồm nhiều literal. Hãy giải thích vai trò của mutex giữa các literal mục tiêu trong quyết định mở rộng planning graph hoặc kết luận thất bại.

- Mở rộng đồ thị: Nếu tại tầng Si đã xuất hiện đủ các literal mục tiêu nhưng vẫn còn ít nhất một cặp mục tiêu mutex với nhau, nghĩa là chúng chưa thể đồng thời đúng. Thuật toán chưa thể trích xuất kế hoạch mà phải tiếp tục mở rộng đồ thị sang tầng tiếp theo (Ai -> Si+1).
- Kết luận thất bại: Nếu đồ thị đã đạt mức ổn định (leveled off, tức Si+1 = Si và các quan hệ mutex không đổi) mà các mục tiêu vẫn chưa xuất hiện đủ hoặc vẫn còn cặp mục tiêu mutex với nhau hoặc trích xuất kế hoạch thất bại, thuật toán kết luận bài toán vô nghiệm và dừng lại.

d) Nêu quy trình tổng quát để trích xuất một kế hoạch từ planning graph sau khi tất cả literal mục tiêu đã xuất hiện ở cùng một mức và không mutex với nhau.

- Bước 1: Bắt đầu tìm kiếm ngược (backward search) từ tầng trạng thái cuối Sn với tập mục tiêu Gn = G.
- Bước 2: Tại tầng hành động An-1 ngay trước đó, chọn một tập con các hành động (bao gồm cả No-op) không có quan hệ mutex với nhau sao cho hiệu ứng thêm (Add list) của chúng bao phủ toàn bộ Gn.
- Bước 3: Đặt tập mục tiêu mới Gn-1 là hợp các tiền điều kiện (Preconditions) của tập hành động vừa chọn, rồi lặp lại Bước 2 lùi dần về tới trạng thái ban đầu S0.
- Bước 4: Nếu tại một tầng bất kỳ không chọn được tập hành động hợp lệ, thực hiện quay lui (backtracking) để thử tổ hợp hành động khác ở tầng sau; nếu thử hết đều thất bại thì mở rộng đồ thị thêm một mức.

— Hết —